Integration of Decision-Making and Motion Planning for Autonomous Driving Based on Double-Layer Reinforcement Learning Framework

Yaping Liao, Guizhen Yu, Peng Chen, *Member, IEEE*, Bin Zhou, and Han Li

***Abstract*—Autonomous driving involves multi-timescale and multi-objective tasks coupled with long-term driving decision and short-term motion planning. However, existing studies tend to investigate them separately or assume they were synchronous and instantaneous, failing to deeply interpret their interconnections in autonomous driving. To address it, this study explored driving decision and motion planning in an integrated manner, and proposed a double-layer reinforcement learning (RL) framework to couple and optimize these two modules. Specifically, on the upper layer of the framework, a trajectory-level reward function associated with safety, efficiency, comfort and decision execution was proposed, and a decision-making model was established based on value function-based deep reinforcement learning (DRL) algorithms. Then, the reward function of the upper layer was taken as the objective function of the lower layer with relevant state constraints. The model predictive control (MPC) method was used to derive the optimal maneuver sequence guided by driving decisions, and the performance evaluation of motion planning was fed back to the decision-making for DRL parameter optimization. Accordingly, a closed-loop mechanism was developed consisting of forward decision-making output guidance and backward motion planning feedback optimization. Then, two-lane and three-lane interactive scenarios were set for framework training and testing. Last, comparative experiments were conducted using NGSIM dataset. The results demonstrated the effectiveness and the enhanced driving performance of the proposed framework in contrast to four benchmarks.**

***Index Terms*—Decision making, motion planning, double-layer framework, autonomous vehicle, deep reinforcement learning (DRL).**

I. Introduction

AUTONOMOUS driving (AD) technologies are shaping the future transportation mode of human beings, which are expected to achieve convenient, efficient and safe travel [1], [2]. It has been well recognized that decision-making and motion planning are the core modules of AD system, which are designed to output appropriate maneuvers for vehicles to react to the dynamic driving environment. The former is responsible for analyzing the current driving environment and making behavioral strategies. Subsequently, the latter outputs the motion maneuvers for autonomous vehicle actuators with the guidance of the driving decision [3].

To date, numerous scholars have been exploring solutions for driving decision and motion planning, which can be mainly divided into three categories: decomposition scheme, end-to-end scheme and integrated scheme. The decomposition scheme, which has desirable interpretability, aims to split the driving decision and motion planning for separate implementation. However, the driving decision and motion planning are strongly coupled in essence. Driving decision defines the vehicle motion planning space and guides the search direction of the motion trajectory. The results of motion planning in turn reflect and affect the performance of driving decision. The design of the driving decision module in the decomposition scheme usually fails to consider the coupling relationship, and can only determine the optimal driving decision based on the instantaneous benefit. The end-to-end scheme takes the state information obtained by the perception module as input, and uses a learning-based policy to derive the expected motion commands. This scheme has efficient computing performance and can also avoid the possible contradiction of optimization objectives among modules. However, the 'black box' end-to-end solution based on deep networks makes the AD system have poor interpretability and adjustability.

To solve these problems, an integrated scheme has been proposed which puts the driving decision and motion planning modules into one model or framework for integrated optimization. This solution considers the coupling relationship between the two modules, making the output of driving decision more reliable and further improving the interpretability of the AD system. Currently, optimization methods and reinforcement learning are the mainstream methods for integrated modeling. The optimization method can simultaneously output the driving decision at the current moment and the corresponding vehicle motion trajectory within a certain time horizon. However, in highly dynamic and complex driving environment, it is constrained by long computation time and local optimization when solving nonlinear and non-convex problems. Reinforcement learning (RL) can solve dynamic programming problems and obtain globally optimal decisions through interactions with unknown environments [4].

Manuscript received 14 July 2023; revised 25 September 2023; accepted 14 October 2023. Date of publication 23 October 2023; date of current version 14 March 2024. This work was supported in part by the National Research and Development Program of China under Grant 2022YFB4703702 and in part by the National Natural Science Foundation of China under Grants 52272327 and 51775016. The review of this article was coordinated by Dr. Bo Yu. *(Corresponding author: Peng Chen.)*

The authors are with the Key Laboratory of Autonomous Transportation Technology for Special Vehicles, Ministry of Industry and Information Technology, School of Transportation Science and Engineering, Beihang University, Beijing 100191, China (e-mail: liaoyapingsk@buaa.edu.cn; yugz@buaa.edu.cn; cpeng@buaa.edu.cn; binzhou@buaa.edu.cn; leehan@buaa.edu.cn).

Digital Object Identifier 10.1109/TVT.2023.3326548

The deep reinforcement learning (DRL) method, formed by combining RL with deep networks, offers the advantages of processing high-dimensional environmental data and efficient computing. In the current DRL-based integrated scheme, the driving decision module only conducts reward evaluations based on the vehicle state at the last moment in time horizon to optimize the driving decision. However, driving decisions inherently involve long-term actions that require a certain time interval to complete. Thus, performing instantaneous evaluations of driving decisions based solely on state points would be inaccurate, thereby impacting the driving performance of the AD system.

To this end, this study established a double-layer DRL framework to investigate decision-making and motion planning in an integrated manner. Based on the environmental perception information, the proposed solution allows for the simultaneous determination of driving decisions for lateral positions at the upper level and maneuvers involving accelerations and steering angles at the lower level in multilane scenarios. The contributions of this study can be summarized as follows:

1) A novel optimization framework has been developed by integrating the DRL driving decision model with the MPC motion planning model. The integrated framework employs a closed-loop mechanism that encompasses both forward output guidance and backward optimal feedback. This enables the DRL driving decision model to adjust and optimize parameters based on the lower-level motion trajectory features.
2) An innovative trajectory-level reward function was designed to guide the parameter optimization of DRL decision-making, with a focus on driving safety, efficiency, comfort, and decision execution. This approach evaluates instantaneous driving decisions based on extracted vehicle trajectory features within a specific time horizon. It enhances the accuracy and effectiveness of DRL decision-making, leading to safer, more efficient, and comfortable driving experiences.
3) An optimization model of motion planning was established by incorporating the proposed trajectory-level reward function into a multi-objective MPC. Instead of implementing safety boundary crafted based on explicit rules, the dynamically constructed driving risk model within the trajectory-level reward function was adopted as safety envelopes. Both minimizing driving risk and matching driving decision commands were considered as optimization objectives of MPC, thereby enabling safe control toward driving decision can be achieved in an efficient and smooth manner.

The rest of this study is organized as follows. The related work is reviewed in Section II. The integrated framework of driving decision and motion planning is introduced in Section III. The development of DRL decision-making model is elaborated in Section IV. Section V establishes the MPC-based motion planning model and the double-layer RL framework. Section VI builds the interactive environment in two-lane and three-lane scenarios to train and test the proposed framework. Last, Section VII concludes this study.

II. Related Work

To date, there have been a significant number of studies conducted on decision-making and motion planning that are reliant on the aforementioned three schemes [5].

A. Decomposition Scheme

In the decomposition scheme, driving decision and motion planning are modeled or investigated separately.

1) Driving Decision: Driving decision methods mainly include rule-based, utility theory-based, game theory-based, Markov process-based and learning-based methods: *a) Rule-based methods* resort to the established if-else driving behavior rules based on certain criteria such as speed expectations and safe distance keeping. Wang et al. [6] designed intelligent decision-making strategies for path planning in autonomous vehicles using a finite state machine (FSM), which including cruise control, car-following, real-time avoidance and early avoidance. *b) Utility theory-based methods* establish utility functions related to safety, efficiency or other indicators by analyzing the relative motion relationship between the ego vehicles and adjacent vehicles on the target lane. These methods make driving decisions to maximize utility. For example, the MOBIL (minimizing overall braking caused by lane changes) method proposed by Kesting et al. [7] uses acceleration as the utility function and minimizes overall braking as the lane-changing criterion. *c) Game theory-based methods* consider the lane changer and the direct follower on the target lane as competitive entities, and determine the optimal outcome by analyzing the cost and benefit of each player in the competition. Exploiting the inherent capability of game theory, Ali et al. [8] developed a mandatory lane-changing model for the traditional environment and extended it for the connected environment. *d) Markov process-based methods* design elements such as states, actions, rewards, and state transition probabilities to make driving decisions that maximize cumulative rewards. Kamrani et al. [9] applied a Markov Decision Process (MDP) framework to explore driving behavior regarding acceleration, deceleration and maintaining speed decisions. In addition, *e) Learning-based methods*, such as support vector machines (SVM) [10] and deep neural network (DNN) [11], have gained popularity. These methods take traffic environment data, such as vehicle trajectories and road geometry, as input and generate driving decisions as output. Decision-making model are trained using extensive empirical data. Based on the current decision-making results, a predicted collision-free trajectory that satisfies vehicle dynamics and road constraints is calculated by the motion planning module.

2) Motion Planning: Typical algorithms for motion planning include graph search, sampling-based methods, interpolation curves, numerical optimization and imitation learning-based approaches: *a) Graph search*, as a practical path planning method, needs to be combined with Adaptive Cruise Control (ACC), S-T graph, and other velocity planning algorithms to avoid obstacles [12], [13]. Its optimization results depend on the density of the graph. *b) Sampling-based methods* constrain vehicle

trajectories by predefining vehicle motion states at sampling points. These methods generate multiple alternative trajectories through the spatio-temporal decoupling of path-velocity or lateral-longitudinal trajectories, and find the optimal trajectories through exhaustive search. They are widely used due to their simplicity, effectiveness and adaptability [14]. However, finding the optimal planned trajectory can be challenging due to the high dimensionality of the sampling space, leading to algorithmic complexity and difficulties in meeting real-time computation requirements. *c) Interpolation curves*, such as polynomial curves, rely on predefined fixed endpoints for modeling [15], [16]. *d) Imitation learning-based methods* aim to achieve motion planning by observing and replicating expert behavior, and have the advantages of online learning and real-time adjustment in practical scenarios. For example, Wang et al. [17] proposed an explainable hierarchical neural network motion planner using imitation learning for urban AD applications. Kuefler et al. [18] applied Generative Adversarial Imitation Learning (GAIL) to model human driving behavior on highways, showing that deep imitation learning could produce realistic highway driver models. While this method does not require fixed endpoints as input, it does require exemplary behavior data. *e) Numerical optimization methods*, such as model predictive control (MPC) [19], [20], transform the motion planning problem into an optimization problem by defining an objective function and constraints. These methods search for the optimal trajectory within a finite time horizon. However, the decomposition scheme primarily focuses on the optimization modeling of individual modules, paying less attention to the relationships and interconnections between these two modules. This may even lead to conflicts in their optimization objectives, which can impact the driving performance of the AD system. Moreover, this scheme only provides immediate benefits and fails to consider the impact of driving decisions on long-term benefits in the future. This deficiency may result in local optimum decision-making rather than global optimum decision-making over time.

B. End-to-End Scheme

Regarding the end-to-end scheme, Nvidia developed an end-to-end driving model utilizing a convolutional neural network (CNN) based on human driver steering maneuver data [21]. Codevilla et al. [22] introduced a command-conditional imitation learning approach for learning from expert demonstrations encompassing low-level controls and high-level commands. The resulting driving policy functions could manage sensorimotor coordination, acting as a chauffeur to make decisions for collision avoidance. Tang et al. [23] proposed a Soft Actor-Critic-based controller that seamlessly integrates the decision-making layer and motion planning layer in an end-to-end fashion. With input from environment perception information, the controller was able to output continuous control parameters for acceleration and steering angle. Li et al. [24] proposed a lane change decision-making framework based on DRL to produce risk-aware steering driving strategies with the lowest expected risk for AD systems. The end-to-end solution can promptly respond to perceived information, significantly enhancing the learning efficiency and performance of AD systems, while also handling more complex driving scenarios. However, this solution suffers from poor interpretability, and safety performance cannot always be guaranteed.

C. Integrated Scheme

At present, there has been increasing studies exploring integrated solutions, primarily divided into optimization-based and deep reinforcement learning-based methods.

1) Optimization-Based Approaches: In an integrated approach presented by Hang et al. [25], Stackelberg Game theory was applied to address driving decision-making and speed planning, while MPC was employed to plan the path for autonomous vehicles. Although this approach achieved a closed-loop iterative optimization process, it made an assumption that obstacle vehicles only engage in acceleration or deceleration behavior and do not change lanes at high speeds, which does not align with real-world scenarios. Liu et al. [26] proposed an optimization-based integrated decision-making and control scheme for urban AD, in which the global path planned by A$*$ algorithm was used as the reference trajectory, and the lower level used optimization algorithm for motion control. This method achieved simultaneous driving decision-making and motion control by designing potential field functions. However, the planning of reference trajectories failed to account for dynamic changes in traffic conditions. This could lead to unreasonable constraints when a vehicle needs to change lanes to pursue safety or efficiency, ultimately affecting the vehicle's overall driving performance.

2) Deep Reinforcement Learning-Based Approaches: Lu et al. [27] proposed a hierarchical RL approach for decision-making and motion planning, where a kernel-based least-squares policy iteration algorithm with uneven sampling and pooling strategy (USP-KLSPI) was presented for solving the decision-making problem and a dual heuristic programming (DHP) algorithm was used to address the motion-planning problem. Although the framework facilitates interaction between decision-making and motion planning, the decision space is characterized by discrete acceleration commands, such as $at = \{Slow, Keep, Acc\}$, which is inconsistent with reality of longitudinal acceleration being a continuous variable for vehicle control. Hoel et al. [28] utilized DRL and the Intelligent Driver Model (IDM) to establish a decision-making model within a custom simulation environment, managing speed and lane-changing decisions for a truck-trailer combination. The output of this model includes handcrafted discrete acceleration commands, assuming that the lane change process is instantaneous and disregarding the lane change execution process. Li et al. [29] employed DRL to integrate lane-changing driving decisions and trajectory planning to help AD systems gain speed advantages while maintaining safety. Compared to previous methods, this approach can guide vehicles to reduce travel time and perform high-level driving maneuvers. However, the path planning in the integrated framework relies on a quintic polynomial method with fixed endpoints, limiting the flexibility of lateral vehicle motion and making it challenging to determine fixed endpoints for each practical path planning scenario. Furthermore, the

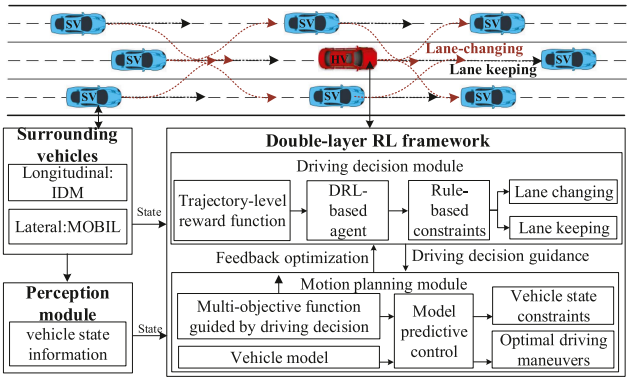

Fig. 1. Integrated framework of driving decision and motion planning for autonomous vehicle.

above DRL integrated frameworks solely conduct instantaneous reward evaluations on driving decisions based on the vehicle state points at the end of each decision-making time horizon. This overlooks the fact that driving decision inherently involves long-term actions. Therefore, after each DRL driving decision, the features of all vehicle state points throughout the entire decision-making time horizon should be extracted for instantaneous reward evaluation.

III. Integrated Framework of Driving Decision and Motion Planning

AD on multi-lanes is essentially a complex and challenging task that necessitates a series of maneuvers to interact with multiple vehicles and ensure driving safety. Fig. 1 illustrates a diagram of a multi-lane driving scenario, featuring two types of vehicles: the host automated vehicle (HV) and the surrounding environmental vehicles (SV). The integrated framework of driving decision and motion planning for HV is also presented in Fig. 1.

For the perception module, it was assumed that HV could accurately obtain the position, speed and acceleration information for itself and the SVs through the sensing system and V2X communication.

Subsequently, within the integrated framework, the decision-making model offered three options for lateral motion: changing to the left lane, lane keeping and changing to the right lane. For SVs, the MOBIL algorithm was used to emulate the lane-changing decisions of human drivers, while IDM was employed to control their longitudinal motion. By resorting to MOBIL and IDM, SVs behaved as agents to interact with one another. For HV, a DRL-based approach was employed to make discrete decisions by referring to the designed trajectory-level reward function. With the driving decision as guidance, the motion planning model determined the optimal acceleration and steering action sequence within the prediction horizon. Furthermore, the motion planning model served to update the state and provide prior knowledge to evaluate the executed decision in advance. Moreover, the actual trajectory features of the HV were fed back into the DRL gradient optimization process to obtain the parameters of the decision-making model.

TABLE I
Description of Notations

Notation	Description
$\boldsymbol{S}(t)$	Environmental state vector
$\boldsymbol{S}_e(t)$	State vector of the ego vehicle
$\boldsymbol{S}_i(t)$	State vector of the surrounding vehicles distributed in potential positions i
$a(t)$	Output actions of driving decision-making model
$\boldsymbol{\Gamma}(t_d)$	Optimal trajectory corresponding to driving decision $a(t_d)$
$R(t_d)$	Discounted cumulative reward
$r(t_d)$	Trajectory-level reward function of driving decision-making model
$p_{x,i}(t)$	X-position of the vehicle i
$p_{y,i}(t)$	Y-position of the vehicle i
$p_{y,road}(t)$	Y-position of the road boundary i
$v_{x,i}(t)$	Longitudinal velocity of the vehicle i
$v_{y,i}(t)$	Lateral velocity of the vehicle i
$\phi_i(t)$	Yaw angle of the vehicle i
v_{desire}	The desired velocity
A	Action space of driving decision-making model
$\tilde{A}$	Feasible action space of driving decision-making model
$\boldsymbol{J}(\cdot)$	Reward function for evaluating the vehicle state at each timestep
$U_{vehicle}$	Driving risk of surrounding vehicles
U_{static}	Driving risk generated by vehicle shape
$U_{kinematic}$	Driving risk generated by vehicle kinematic features
U_{road}	Driving risk generated by road boundary
$C(\cdot)$	Comfort reward function
$V(\cdot)$	Efficiency reward function
$E(\cdot)$	Decision execution reward function
$Q(\cdot)$	Q-value function
$\hat{Q}(\cdot)$	Target Q-value function
$\mathbf{M}$	Experience replay buffer
N	Number of samples from the experience replay buffer
$\boldsymbol{\theta}$	Weights of Q-value function
$\hat{\boldsymbol{\theta}}$	Weights of target Q-value function
$V(\cdot)$	State function
$A(\cdot)$	Advantage function that evaluates the advantage after taking action
N_a	Capacity of action space
$\boldsymbol{\mu}$	Vehicle motion maneuver
μ_{lo}	Vehicle acceleration maneuver
μ_{la}	Vehicle steering angle maneuver
$\boldsymbol{f}_{ego}(\cdot)$	State update model of the ego vehicle
$\boldsymbol{f}_{pred}(\cdot)$	State prediction model of surrounding vehicles
$UT_i(t)$	Utility of a lane change
$\Delta p_{des,i}$	Desired relative distance of the surrounding vehicle i
$\Delta p_{x,i}$	Relative distance of the surrounding vehicle i and its front vehicle
Δv_i	Relative velocity of the surrounding vehicle i and its front vehicle

The decision-making and motion planning modules were integrated into a hierarchical RL optimization framework, referred to as the double-layer RL framework hereafter. This framework offers technical support for AD in dynamic traffic scenarios. Specific details are provided below, and the notations involved are summarized in Table I.

IV. Development of Driving Decision-Making Based on Deep Reinforcement Learning

RL is an important branch of machine learning used to solve problems where agent learns policy to maximize returns or achieve specific goals through interacting with its environment. DRL combines the strengths of deep learning in data feature

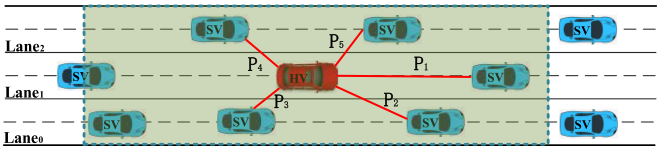

Fig. 2. Diagram of potential positions in multi-lane scenario.

extraction and reinforcement learning in decision-making. In policy optimization, DRL considers not only the local reward at the current moment but also the global reward over the entire planning horizon. Thus, this study utilized DRL to tackle the decision-making challenges by autonomous vehicles.

A. Formulation of DRL Driving Decision Model

The decision-making problem can be described as the standard Markov Decision Process (MDP) in RL. Assume that the duration of each driving decision is denoted as $\Delta t'$, which is λ times the timestep Δt of vehicle movement, i.e., $\Delta\ t' = \lambda \Delta t$. When receiving the environmental state $\boldsymbol{S}(t_d)$ at each decision-making timestep t_d, DRL utilizes the designed networks to determine the optimal driving decision $a(t_d)$ as an instruction of the lower-level motion planning module. Then, the motion planning module outputs the optimal trajectory $\boldsymbol{\Gamma}(t_d)$ for state updating. At each time interval of $\Delta t'$, the designed reward function is used to calculate the reward value $r(t_d)$ based on the actual trajectory $\boldsymbol{\Gamma}^*(t_d)$ and evaluate the instantaneous driving decision$a(t_d)$. Then, the environmental state $\boldsymbol{S}(t_d + \Delta t')$ at the next timestep $t_d + \Delta t'$ is obtained and input into DRL model to make the next driving decision $a(t_d + \Delta t')$. Through performing a sufficient number of iterations involving interaction between the DRL model and the driving environment, the DRL networks learn the optimal parameters $\boldsymbol{\theta}$ with the objective of maximizing the discounted cumulative reward $R\left(t_d\right) = \sum_{k=0}^{\infty} \gamma^k r(t_d + k \cdot \Delta t')$, where the discount factor $\gamma \in (0,\ 1]$ weights the importance of rewards within the time interval $[t_d, t_d + k \cdot \Delta t']$. The learned model becomes capable of evaluating the long-term reward of each alternative driving decision based on the input state.

B. RL Elements

To solve the MDP problem of driving decisions, specific RL elements, including state, action, reward function, network architecture are designed as follows.

1) State Design: The state represents the observation of the environment of the ego vehicle within the defined perception range $[-L,\ L]$. The state space can be defined as:

$$\boldsymbol{S}\left(t\right) = \left[\boldsymbol{S}_e\left(t\right), \left\{\boldsymbol{S}_i\left(t\right)\right\}_{i=1}^{n}\right]^T \tag{1}$$

where n refers to the number of potential positions. Fig. 2 depicts a diagram of potential positions in a multi-lane scenario including $(P_1,\ P_2, \ldots, P_2)$.

The state vector $\boldsymbol{S}_e(t)$ is defined by the physical state of the ego vehicle:

$$\boldsymbol{S}_e = \left[p_{x,\,0}\left(t\right), p_{y,0}\left(t\right), v_{x,0}\left(t\right), v_{y,0}\left(t\right), \phi_0\left(t\right)\right] \tag{2}$$

Then, the relative states of surrounding vehicle i, including relative positions $\Delta p_{x,i}(t)$, $\Delta p_{y,i}(t)$ and relative velocities $\Delta v_{x,i}(t)$, $\Delta v_{y,i}(t)$ are extracted to form the state vector $\boldsymbol{S}_i(t)$:

$$\begin{aligned}&\boldsymbol{S}_i(t)\\ &= \left[I\left(t\right), \Delta p_{x,i}\left(t\right), \Delta p_{y,i}\left(t\right), \Delta v_{x,i}\left(t\right), \Delta v_{y,i}\left(t\right), \phi_i\left(t\right)\right]\end{aligned} \tag{3}$$

where, the relative positions are calculated as $\Delta p_{x,i}\ \left(t\right) = p_{x,i}\ \left(t\right) - p_{x,0}(t)$, $\Delta p_{y,i}\ \left(t\right) = p_{y,i}\ \left(t\right) - p_{y,\,0}(t)$. The relative velocities are calculated by $\Delta v_{x,i}\ \left(t\right) = v_{x,i}\ \left(t\right) - v_{x,0}(t)$, $\Delta v_{y,i}\ \left(t\right) = v_{y,i}\ \left(t\right) - v_{y,0}(t)$. $I_i = \{0,\ 1\}$ is a binary variable that indicates whether the surrounding vehicle in the potential position P_i exits or not. If $I_i =\ 0$, the corresponding state information will be filled with 0.

2) Action Design: In multi-lane scenarios, the driving decisions include three discrete options: changing to the left lane, lane keeping, and changing to the right lane. This study established an action space based on the y-position along the longitudinal centerline corresponding to different target lanes:

$$a_j \in A = \left\{p_{y,lane_0}, p_{y,lane_1}, p_{y,lane_2}\right\} \tag{4}$$

where $a_0 = p_{y,lane_0}$, $a_1 = p_{y,lane_1}$, and $a_2 = p_{y,lane_2}$ denotes that the target lane is lane_0, lane_1 and lane_2, respectively, as shown in Fig. 2. For scenarios with more lanes, the action space will be expanded according to the number of lanes.

In the process of DRL policy exploration and exploitation, the deep learning network may occasionally produce undesirable and risky actions, potentially resulting in vehicle collisions or violations of traffic rules. Even if penalties can be incorporated into the reward function to discourage these occurrences, the black-box nature and uncontrollability of deep network make the feasibility of output action not guaranteed. To tackle it, this study designed a rule-based constraint for extracting a feasible action space, denoted as $\tilde{A}$:

1) For multi-lane scenarios, the ego vehicle is restricted to selecting the adjacent lane or the current lane for decision-making.
2) Actions that prevent the lower-level motion planning model from generating a safe trajectory are excluded from the feasible action set $\tilde{A}$.
3) When there are no preceding vehicles or stationary roadblocks within the detection range of the ego vehicle, the vehicle shall keep driving in the current lane.

Then, the feasible action space $\tilde{A}$ can be expressed as:

$$\tilde{A} = \left\{a \,|a \in A\ \&\ a \text{ subjects to constraints}\right\} \tag{5}$$

Actions that fall outside the feasible action space $\tilde{A}$ will not be permitted. In such case, as an alternative approach, the action with the maximum Q-value within the feasible action space $\tilde{A}$ will be selected as the driving decision result of DRL.

3) Reward Function Design: In the DRL framework, the reward function plays a critical role in the effectiveness and convergence of model training. As aforementioned, most studies only used the reward function to evaluate the vehicle state at the end of the driving decision interval. However, it's important to note that a driving decision is essentially a long-term action,

and its performance is reflected by the lower-level trajectory. Trajectories with the same vehicle state features at the end may have different driving experience during the decision interval. Thus, relying solely on the vehicle state features at the end of the decision interval may not accurately evaluate the driving decision.

To address this issue, a trajectory-level reward function was proposed for evaluating the driving decision. It calculates the reward value based on the states $[\boldsymbol{S}(t_d), \boldsymbol{S}(t_d+\Delta t), \ldots, \boldsymbol{S}(t_d+\Delta t')]$ extracted from the vehicle trajectory within $[t_d, t_d+\Delta t']$:

$$r(t_d) = \sum_{t=0}^{\Delta t'} \boldsymbol{J}\left(\boldsymbol{S}\left(t_d+t\right)\right) \tag{6}$$

In this paper, the reward function $\boldsymbol{J}(\cdot)$ takes into account safety, efficiency, comfort and decision execution.

a) Safety reward: The safety reward cannot be disregarded even if rule-based constraints have been applied to extract feasible action space. The reason is that the rule-based constraints are employed solely to extract feasible action spaces, which contain driving decision actions that can be considered as potential output. However, the optimal driving decision must still be determined from the feasible action space through the reward mechanism to guide the lower-level motion planning.

In this reward mechanism, given the current vehicle state $\boldsymbol{S}(t_d+t)$, the driving risk generated by the road lane and surrounding vehicles is used to represent the safety reward. To enable the DRL model to output driving decisions that guide the ego vehicle along a trajectory with minimal risk, an exponential function was employed to establish the driving risk model. This model quantifies the risk level of vehicle trajectories in the driving environment and serves as a safety objective function in the motion planning layer described in Section V-A below. This function is equivalent to a safety envelope constraint for trajectory planning, thus eliminating the need for the imposition of explicit rule-based safety boundaries.

The driving risk $U_{vehicle}$ of surrounding vehicle posing to the ego is determined by the vehicle shape and kinematic features, represented by U_{static} and $U_{kinematic}$ respectively.

$$U_{vehicle}(\boldsymbol{S}\left(t_d+t\right)) = U_{static} + U_{kinematic} \tag{7}$$

$$U_{static}(\boldsymbol{S}\left(t_d+t\right)) = k_1 \cdot \sum_{i=1}^{n} e^{-\left(\frac{\Delta p_{x,i}(t_d+t)^2}{l^2}+\frac{\Delta p_{y,i}(t_d+t)^2}{w^2}\right)^2} \tag{8}$$

where l and w represent the length and width of the vehicle, respectively, k_1 is the weight coefficient.

$$U_{kinematic}(\boldsymbol{S}\left(t_d+t\right)) = k_2 \cdot \sum_{i=1}^{n} \frac{e^{-\left(\frac{\Delta p_{x,i}(t_d+t)^2}{\left[k_v \cdot \Delta v_{x,i}(t_d+t)^2\right]}+\frac{\Delta p_{y,i}(t_d+t)^2}{w^2}\right)^2}}{1+e^{D_{v,i}\cdot(-\Delta p_{x,i}(t_d+t)-\alpha\cdot l\cdot D_{v,i})}} \tag{9}$$

$$D_{v,i}(\boldsymbol{S}\left(t_d+t\right)) = \operatorname{sgn}\left(-\Delta v_{x,i}\left(t_d+t\right)\right) = \begin{cases} -1, & v_{x,0}\left(t_d+t\right) < v_{x,i}\left(t_d+t\right) \\ 0, & v_{x,0}\left(t_d+t\right) = v_{x,i}\left(t_d+t\right) \\ 1, & v_{x,0}\left(t_d+t\right) > v_{x,i}\left(t_d+t\right) \end{cases} \tag{10}$$

The dynamic risk is determined by the relative velocity between the obstacle and the ego vehicle. In the same driving direction, when the velocity of the leading vehicle is higher than that of the ego vehicle, it indicates that the leading vehicle is gradually moving away from the ego. In such cases, the risk distribution generated by the leading vehicle should also be skewed away from the ego. Otherwise, when the velocity of the leading vehicle is lower than the ego, it indicates that the ego is gradually approaching the leading vehicle, and the risk distribution generated by the leading vehicle should be skewed toward the ego. Thus, the relative velocity was utilized to structure the kinematic risk distribution $U_{kinematic}$, as shown in (10). $D_{v,i}$ denotes the relative state of vehicle i to the ego, which is used to determine the orientation of risk distribution. k_v, α are constants satisfying $k_v > 0$, $0 \le \alpha \le 1$.

In addition to the surrounding vehicles, road boundaries also pose a certain risk to prevent vehicle invasions. Its risk distribution is expressed as:

$$U_{road}(\boldsymbol{S}\left(t_d+t\right)) = k_3 \cdot \sum_{j=1}^{2} e^{-\left(p_y(t_d+t)-p_{y,road_i}(t_d+t)\right)^2} \tag{11}$$

Then, the safety reward is obtained by:

$$U\left(\boldsymbol{S}\left(t_d+t\right)\right) = -U_{vehicle} - U_{static} - U_{road} \tag{12}$$

b) Efficiency reward: The efficiency reward function was designed to evaluate driving efficiency and encourage the model to make decisions guiding the vehicle to pursue the desired velocity whilst ensuring safety. The function is expressed as:

$$V(\boldsymbol{S}\left(t_d+t\right)) = - k_4 \cdot \left(\operatorname{sqrt}\left(v_{x,0}\left(t_d+t\right), v_{y,0}\left(t_d+t\right)\right) - v_{desire}\right)^2 \tag{13}$$

where sqrt$(\cdot)$ is a square root function to calculate the actual velocity of the ego.

c) Comfort reward: The comfort reward function was used to evaluate driving comfort and punish aggressive motion maneuvers:

$$C(\boldsymbol{S}\left(t_d+t\right)) = -\left|\boldsymbol{\mu}\boldsymbol{\Theta}\boldsymbol{\mu}^{\mathrm{T}}\right| \tag{14}$$

where $\boldsymbol{\mu} = [\mu_{lo}, \mu_{la}]$, $\boldsymbol{\Theta} = diag(k_5, k_6)$ represents the weight matrix composed of weight coefficients k_5 and k_6.

d) Decision execution reward: The decision execution reward was employed to assess the executability of DRL driving decisions during the model training process. Executability is quantified by the deviation between the actual trajectory and the target lane $\widehat{p}_{y,lane}$. A larger deviation indicates lower executability, resulting in a reduced decision execution reward. Then, during the training process, incorporating the decision

execution reward encourages the DRL driving decision module to produce driving decisions with high executability while maintaining safety, efficiency, and comfort.

Its expression is shown as:

$$E(\boldsymbol{S}(t_d+t)) = -\left(p_{y,0}(t_d+t) - \widehat{p}_{y,lane}(t_d)\right)^2 \tag{15}$$

Based on the rewards of safety, efficiency, comfort and decision execution, the total reward can be expressed as:

$$\begin{aligned}\boldsymbol{J}(\boldsymbol{S}(t_d+t)) = U(\boldsymbol{S}(t_d+t)) + V(\boldsymbol{S}(t_d+t)) \\ + C(\boldsymbol{S}(t_d+t)) + E(\boldsymbol{S}(t_d+t))\end{aligned} \tag{16}$$

4) Training Algorithm: The DRL algorithms can be divided into two categories: policy-based and value-function based, which are used to solve continuous and discrete action space problems, respectively. Because the driving decision model in this study outputs discrete values, the value-function based Deep Q-network (DQN) was used to train the driving decision model. Three DQN algorithms, including basic DQN, DDQN and Dueling DQN, are described below.

a) Basic deep Q-network (DQN): DQN is suitable for solving MDP problems with discrete action space, which has been effectively applied to games, robot control, traffic control and other fields. It has two neural networks $Q(s,a|\boldsymbol{\theta})$ and $\widehat{Q}(s,a|\widehat{\boldsymbol{\theta}})$ to approximate the Q-value function. $Q(s,a|\boldsymbol{\theta})$ is responsible for generating the trajectory sample (s_t, a_t, r_t, s_{t+1}) that are stored in the experience replay buffer $\mathbf{M}$. Subsequently, a minibatch of samples is randomly extracted from $\mathbf{M}$ to optimize the network parameter. $\widehat{Q}(s,a|\widehat{\boldsymbol{\theta}})$ calculates the temporal difference (TD) error for updating the $Q(s,a|\boldsymbol{\theta})$:

$$L = \frac{1}{N}\sum_i \left(Y_i^{DQN} - Q(s_i,a_i|\boldsymbol{\theta})\right)^2 \tag{17}$$

$$Y_i^{DQN} = r_i + \gamma \max_{a_{i+1}} \widehat{Q}\left(s_{i+1}, a_{i+1} \middle| \widehat{\boldsymbol{\theta}}\right) \tag{18}$$

b) Double deep Q-network (DDQN): DQN uses the maximum target Q-value for parameter updating, which makes the model likely to arise overestimation problem in action selection. Thus, DDQN is proposed to solve this overestimation problem. It uses $Q(s,a|\boldsymbol{\theta})$ to select target actions, but $\widehat{Q}(s,a|\hat{\boldsymbol{\theta}})$ to evaluate these actions in TD error calculation:

$$L = \frac{1}{N}\sum_i \left(Y_i^{DDQN} - Q(s_i,a_i|\boldsymbol{\theta})\right)^2 \tag{19}$$

$$Y_i^{DDQN} = r_i + \gamma Q\left(s_{i+1}, \underset{a_{i+1}}{arg\max}\, \widehat{Q}(s_{i+1},a_{i+1}|\boldsymbol{\theta}) \middle| \widehat{\boldsymbol{\theta}}\right) \tag{20}$$

c) Dueling deep Q-network (DulDQN): DQN and DDQN both adopt a single-stream output to compute the Q-value with the joint determination of state and action. In certain case the action has no effect on the environment, however, the state still determines the Q-value. DulDQN splits the single stream output of DQN's Q-function into two stream outputs: one for the state value and the other for the action advantage.

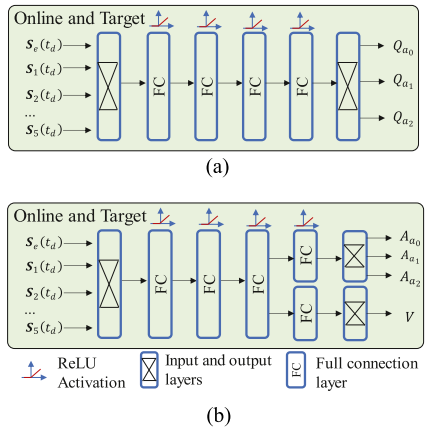

Fig. 3. Network architecture of DQN algorithms. (a) DQN and DDQN. (b) Dueling DQN.

$$Q(s,a|\boldsymbol{\theta}) = V(s|\boldsymbol{\theta}) + A(s,a|\boldsymbol{\theta}) \tag{21}$$

Considering that (21) exists an unidentifiable problem that the V and A splits from the Q-value are not unique, the constraint of $E_{a\sim\pi(s)}[A(s,a|\boldsymbol{\theta})] = 0$ is used to address this problem. Then, the expression of the Q-function is updated to:

$$Q(s,a|\boldsymbol{\theta}) = V(s|\boldsymbol{\theta}) + \left(A(s,a|\boldsymbol{\theta}) - \frac{1}{N_a}\sum_{a'} A(s,a'|\boldsymbol{\theta})\right) \tag{22}$$

5) Network Architecture: In DQN and DDQN, online value function and target value function have the same network architecture. Both consist of a 35-dimension input layer, four full connection layers with 50 hidden nodes, and a three-dimension output layer. Different from DQN and DDQN, the network structure of Dueling DQN consists of one 35-dimension input layer, three hidden layers, one hidden layer for calculating the state value, one hidden layer for calculating the action advantage value, one output layer for calculating the state value and one output layer for calculating the action advantage value. The activation function of all hidden layers is the ReLU function. Their network architectures are shown in Fig. 3.

V. Double-Layer RL Framework for Decision-Making and Motion Planning

In this section, the MPC-based motion planning model was established with the designed objective function and constraints. Subsequently, it was embedded into the DRL driving decision framework to form a double-layer RL framework integrating driving decision and motion planning. Finally, the specific implementation of the double-layer RL framework was summarized.

A. MPC-Based Motion Planning Model

In each timestep t, guided by the driving decision $a^* \in \tilde{A}$ output by DRL, the lower-level motion planning was designed

Algorithm 1: Double-Layer RL Framework.

Initialize value function Q, $\widehat{Q}$ with random parameters $\boldsymbol{\theta}$, $\widehat{\boldsymbol{\theta}}$, replay buffer M, exploration probability ε
For each episode **do**
  Initialize environment and $d(t_d) = False$
  Receive initial observation state $\boldsymbol{S}(t_d)$
  While not $d(t_d)$:
    With probability ε select a random action $a(t_d)$
    Otherwise select driving decision with Q:
      $a(t_d) = \arg\max_{a(t_d)} Q(S(t_d), a; \boldsymbol{\theta}),\ a \in \tilde{A}$
    For t in $[t_d, t_d + \Delta t']$ **do**
      Calculate vehicle motion maneuver $\boldsymbol{\mu}(t)$ with the driving decision $a(t_d)$ using (23)
      Update vehicle state using the environment model and observe $\boldsymbol{S}(t)$
      Calculate reward for state point using (16)
      If $\boldsymbol{S}(t)$ is terminal or task failed **do**
        $d(t_d) = True$
    Calculate the trajectory reward $r(t_d)$ using (6)
    Observe $[\boldsymbol{S}(t_d), a(t_d), r(t_d), \boldsymbol{S}(t_d + \Delta t'), d(t_d)]$ and store to M
    Perform parameter optimization at $\boldsymbol{\theta}$ and $\widehat{\boldsymbol{\theta}}$ using DQN series algorithm
  Until convergence

to determine the motion control quantities by solving a constrained problem within a finite horizon T. The objective was to minimize the distance to the target lane, driving risk, travel time and comfort. To ensure the integration compatibility and target consistency of the driving decision and motion planning, the reward function $\boldsymbol{J}(\boldsymbol{S})$ designed in Section IV was used as the objective function of the motion planning model:

$$\boldsymbol{\mu}(t) = \arg\max_{\boldsymbol{\mu}(t)} \sum_{k=0}^{T-1} \boldsymbol{J}(\boldsymbol{S}(k|t)) \tag{23}$$

In $\boldsymbol{J}(\boldsymbol{S})$, the decision execution reward plays a role in promoting the driving decision to guide the vehicle motion planning. By taking the reward function $\boldsymbol{J}(\boldsymbol{S})$ as the objective function, the motion planning model controls the ego vehicle to move towards the target lane along the optimal trajectory in terms of safety, efficiency and comfort. The safety objective function here is the dynamically constructed driving risk model mentioned in Section IV-B3 above, which serves as a safety envelope with soft constraints.

During driving, the vehicle state should be subject to the following constraints:

$$\boldsymbol{S}_e(i+1|t) = \boldsymbol{f}_{ego}(\boldsymbol{S}_e(i|t), \boldsymbol{\mu}(i|t)) \tag{24}$$

$$\boldsymbol{S}_i(i+1|t) = \boldsymbol{f}_{predict}(\boldsymbol{S}_i(i|t)) \tag{25}$$

$$\boldsymbol{\mu}_{\min} \le \boldsymbol{\mu}(i|t) \le \boldsymbol{\mu}_{\max} \tag{26}$$

$$\Delta\boldsymbol{\mu}_{\min} \le \boldsymbol{\mu}(i+1|t) - \boldsymbol{\mu}(i|t) \le \Delta\boldsymbol{\mu}_{\max} \tag{27}$$

$$p_{y,road_1}(i|t) \le p_{y,0}(i|t) \le p_{y,road_2}(i|t) \tag{28}$$

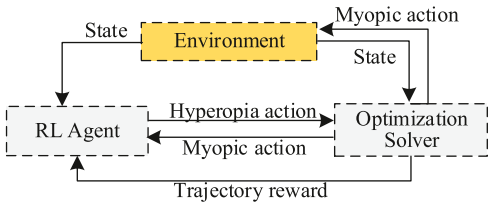

Fig. 4. Design mechanism of double-layer RL model.

$$\boldsymbol{S}_e(0|t) = \boldsymbol{S}_e(t), \boldsymbol{S}_i(0|t) = \boldsymbol{S}_i(t), \boldsymbol{\mu}(0|t) = \boldsymbol{\mu}(t) \tag{29}$$

where (24) and (25) represent the state transition constraints of ego and other vehicles, respectively. $\boldsymbol{\mu}_{\min}$ and $\boldsymbol{\mu}_{\max}$ in (26) are the matrices of the lower and upper bounds of the control inputs, respectively. $\Delta\boldsymbol{\mu}_{\min}$ and $\Delta\boldsymbol{\mu}_{\max}$ in (27) are the matrices of the lower and upper bounds of the control inputs changes, respectively. Equation (28) represents that the y-positions of all optimized states are constrained by the road boundary. The expressions for $\boldsymbol{f}_{ego}$ and $\boldsymbol{f}_{pred}$ are shown in (30) and (31), respectively. The predicted states in (23) are produced by the vehicle kinematics model in (30) and (31), and the initial state $\boldsymbol{S}(0|t)$ is assigned with the current real state $\boldsymbol{S}(t)$.

$$\boldsymbol{f}_{ego} = \begin{bmatrix} p_{x,0} + \Delta t \cdot (v_{x,0}\cos\phi_0 - v_{y,0}\sin\phi_0) \\ p_{y,0} + \Delta t \cdot (v_{x,0}\sin\phi_0 + v_{y,0}\cos\phi_0) \\ \left(\sqrt{v_{x,0}^2 + v_{y,0}^2} + \mu_{lo}\Delta t\right) \cdot \cos\phi_0 \\ \left(\sqrt{v_{x,0}^2 + v_{y,0}^2} + \mu_{lo}\Delta t\right) \cdot \sin\phi_0 \\ \phi_0 + \frac{\sqrt{v_{x,0}^2+v_{y,0}^2}}{L_v} \cdot \tan\mu_{la} \end{bmatrix} \tag{30}$$

$$\boldsymbol{f}_{pred} = \begin{bmatrix} p_{x,i} + \Delta t \cdot (v_{x,i}cos\phi_i - v_{y,i}\sin\phi_i) \\ p_{y,i} + \Delta t \cdot (v_{x,i}\sin\phi_i + v_{y,i}\cos\phi_i) \\ v_{x,i}\cos\phi_i - v_{y,i}\sin\phi_i \\ v_{x,i}\sin\phi_i + v_{y,i}\cos\phi_i \\ \phi_i \end{bmatrix} \tag{31}$$

By solving the above optimization problem, the optimal control maneuver sequence can be obtained as follows:

$$\boldsymbol{\mu}^*(t) = [\boldsymbol{\mu}(0|t), \boldsymbol{\mu}(1|t), \ldots, \boldsymbol{\mu}(T-1|t)] \tag{32}$$

Then, the first control maneuver $\boldsymbol{\mu}(0|t)$ is used to updated the vehicle state, and a new optimization process is initialized in the next timestep $t+1$.

B. Double-Layer RL Framework

The vehicle motion is a hybrid action space problem with constraints, involving both discrete driving decisions and continuous vehicle control maneuvers. Driving decisions can be taken as hyperopia actions, which help guide the optimization direction of vehicle control maneuvers. Vehicle control maneuvers can be referred to as myopic actions, which reflect the long-term performance of driving decisions. Accordingly, this study designed a double-layer RL framework to realize the integrated optimization of upper-layer decision-making and lower-layer motion planning.

As depicted in Fig. 4, the lower optimization solver receives the hyperopia action from the upper layer, employs an optimization method to identify the optimal myopic action, and then returns to the upper layer. The upper-layer RL agent is responsible for selecting appropriate hyperopia actions for learning policy learning, making RL maximize the cumulative reward. The guidance of driving decision can avoid the ineffective search within the lower-layer myopic action space. Since each search for myopic action is optimal, it helps improve the learning efficiency of RL and speed up the model training. The specific implementation of the double-layer RL framework is shown in Algorithm 1.

VI. Training and Validation of the Proposed Framework

In this section, numerical experiments were designed to train and validate the proposed framework. First, the interactive training environment was configured, including relevant parameter settings and environment modeling. Using the designed environment, the proposed framework was trained and tested, and was compared against several benchmark frameworks. Furthermore, the superiority of the proposed framework was demonstrated in empirical driving environment by comparing with the state-of-the-art.

A. Training Environment

1) Environment Design: Two scenarios were set up for model training: two-lane and three-lane scenarios. The ego vehicle randomly selected the initial lane and started with an initial velocity and desired velocity of 20 m/s and 34 m/s, respectively [29], [30]. Regarding the surrounding vehicles, they were positioned along the x-axis of the road, and the inter-vehicle distance was initialized randomly within the range of [25 m, 60 m]. Furthermore, their initial velocities were randomly generated from the range of [17], [25] m/s. To ensure that the ego would experience more lane changes, the desired velocity of the surrounding vehicles was set as 20 m/s, which was lower than that of the ego. To effectively train the framework, the number of surrounding vehicles in the two-lane and three-lane scenarios was randomly selected from the range of [4], [8] and [6], [10], respectively.

The environment of each training episode was initialized with different random seeds. During driving, the surrounding vehicles could adjust their longitudinal and lateral velocities according to the surrounding environment at any time. The ego could accurately perceive the surrounding environment and was expected to explore a driving strategy to reach the destination safely, efficiently and comfortably. When the ego overtook all surrounding vehicles, collided or reached the maximum timestep, the episode ended, and the next episode began.

2) Interactive Environment Modeling: To ensure that each surrounding vehicle behaved as an agent and interacted with other vehicles, MOBIL and IDM were used to control the surrounding vehicles. MOBIL determined lane-changing decisions for surrounding vehicles based on the utility of a lane change:

$$\begin{aligned} UT_i(t) = {} & \mu'_{lo,i}(t) - \mu_{lo,i}(t) \\ & + \eta \cdot \left[\mu'_{lo,n,i}(t) - \mu_{lo,n,i}(t)\right. \\ & \left. + \mu'_{lo,o,i}(t) - \mu_{lo,o,i}(t)\right] \end{aligned} \tag{33}$$

where $\mu'_{lo,i}$ and $\mu_{lo,i}$ represent the accelerations before and after the vehicle i changes lanes. $\mu'_{lo,n,i}$ and $\mu_{lo,n,i}$ represent the accelerations of the new follower before and after the vehicle i changes lanes. $\mu'_{lo,o,i}$ and $\mu_{lo,o,i}$ represent the accelerations of the previous follower before and after the vehicle i changes lanes.

Take the three-lane scenario in Fig. 2 as an example, when the condition in (34) is satisfied, the lane change will be made by the vehicle i:

$$a_i(t) = \begin{cases} p_{y,lane_2}, UT_{i,lane_2} > \Delta A_{th} \text{ and } UT_{i,lane_2} \geq UT_{i,lane_0} \\ p_{y,lane_0}, UT_{i,lane_0} > \Delta A_{th} \text{ and } UT_{i,lane_0} > UT_{i,lane_2} \\ p_{y,lane_1}, \text{otherwise} \end{cases} \tag{34}$$

The conditions for lane-changing decisions in scenarios with other number of lanes can be expanded according to (34).

When the lane-changing decision was made, the corresponding lane-changing path would be planned using the quadratic polynomial method. Then, IDM output the acceleration based on the relative states between the vehicles to control the longitudinal motion of the surrounding vehicle i:

$$\mu_{lo,i}(t) = \mu_{lo,\max}\left[1 - \left(\frac{v_i(t)}{v_{des}}\right)^4 - \left(\frac{\Delta p_{des,i}(t)}{\Delta p_{x,i}(t)}\right)^2\right] \tag{35}$$

$$\Delta p_{des,i}(t) = p_0 + \max\left\{0,\ v_i(t)\cdot\Delta t + \frac{v_i(t)\cdot\Delta v_i(t)}{2\sqrt{\mu_{lo,\max}\cdot\mu_{lo,c}}}\right\} \tag{36}$$

3) Parameters Setting: The relevant parameters setting of DRL, reward function and environment model is presented in Table II. The duration of each driving decision is 0.5 s. During this period, the motion planning module updates the vehicle states once per 0.1 s time step to control the vehicle driving, and then starts to receive the next driving decision command. The parameters of reward functions were obtained through trial-and-error, taking into account human driving habits and lane structure, and they could be adjusted according to different driving styles [31], [32].

4) Training System Device: The entire procedure was conducted on a desktop computer with a 3.60 GHz i7-9700K CPU, 16 GB of RAM and NVIDIA GeForce GTX 1650/PCIe/SSE2 running on Ubuntu 18.04.6 LTS. We implemented the program using Python 3.7 and the neural network using Tensorflow-gpu 1.14.0.

B. Training of DRL Frameworks

The double-layer RL frameworks formed by three DQN decision-making algorithms and the optimization-based motion planning model were referred to as DQN + OP, DDQN + OP and

TABLE II
HYPERPARAMETERS SETTING

Model	Hyperparameters	Values
DRL	Maximum training episode	6000
	Duration of driving decision, $\Delta t'$	0.5
	Vehicle movement timestep, Δt	0.1
	Learning rate	0.001
	Replay buffer size	50000
	Batch size of training samples	32
	Discount factor, $\boldsymbol{\gamma}$	0.99
	Maximum timestep for episode	500
	Target network update frequency	50
Reward function	Vehicle length (m), l	4.8
	Vehicle width (m), w	2.0
	Shape coefficient, k_v	6
	Constant coefficient, α	0.9
	Weight coefficient $k_{i=1,\ldots,6}$	(10,6,6, 0.02, 0.05,10)
Motion planning model	Lower bound of the control inputs (m/s^2, rad), $\boldsymbol{\mu}_{min}$	[-3, -0.1]
	Upper bound of the control inputs (m/s^2, rad), $\boldsymbol{\mu}_{max}$	[3, 0.1]
	Lower bound of the control inputs change	[1, 0.05]
	Model parameter, η	0.15
Environment model	Decision threshold, ΔA_{th}	0.4
	Maximum acceleration, $\mu_{lo,max}$	3
	Comfortable braking deceleration, $\mu_{lo,c}$	1.5
	Minimum distance, p_0	2

DulDQN + OP, respectively. Frameworks with different reward functions were also distinguished. For example, DulDQN + OP with point-level reward function and trajectory-level reward function were represented as DulDQN + OP1 and DulDQN + OP2, respectively. The point-level reward function refers to a function that only calculates the reward for the ending-time vehicle states in the time horizon of each driving decision. To verify the effectiveness of the proposed frameworks and trajectory-level reward function, the following frameworks were trained and compared.

1) *DQN + OP2, DDQN + OP2 and DulDQN + OP2 frameworks:* composed of DQN + OP, DDQN + OP and DulDQN + OP frameworks and trajectory-level reward function, respectively.
2) *DulDQN + OP1 framework:* composed of DulDQN + OP framework and point-level reward function.
3) *DulDQN + Poly framework:* composed of DulDQN algorithm, trajectory-level reward function and polynomial-based motion planning algorithm. For the polynomial-based motion planning algorithm, a fifth-degree polynomial function and IDM were adopted for path planning and speed generation, respectively.

All DRL frameworks were trained for 6000 episodes in each run, and training results related to their learning performance were extracted and compared, including overall reward, safety reward, efficiency reward and decision execution reward. A rolling window with 100 episodes was used to smooth the obtained training rewards for identifying the overall tendency of one training run. As shown in Fig. 5, multiple runs were performed to verify the robustness of the models, with the average rolling reward (ARR) and fluctuating amplitudes illustrated using solid lines and shaded areas, respectively.

Fig. 5(a) and (b) present the training results in two-lane and three-lane scenarios, respectively. It can be observed that the DulDQN + OP2 framework achieves the best learning performance. Its overall ARRs in two scenarios are approximately −533 and −687, respectively. The ARRs for safety, efficiency, comfort and decision execution are approximately (−148, −198 −11, −176) and (−151, −350, −18, −168), respectively. Note that the overall ARR of DulDQN + OP2 in the three-lane scenarios is lower than that in the two-lane scenarios. Such difference can be primarily attributed to driving efficiency. In the three-lane scenarios, the ego vehicle is subject to interacting with more surrounding vehicles distributed in three lanes. To maintain driving safety, the ego vehicle was supposed to reduce its driving velocity to a certain extent, thus reducing the driving efficiency rewards.

For DQN + OP2 and DDQN + OP2, the overall ARRs after convergence are −610 and −566 in the two-lane scenarios, respectively, which are slightly lower than those for DulDQN. By comparison, in the three-lane scenarios, the difference of the overall ARRs after convergence among three frameworks becomes significant. This is because DulDQN helps improve the over-evaluation problem for the state value by the other two DRL algorithms.

The overall ARR of DulDQN + OP1 in the two-lane scenarios is about −562, with safety, efficiency, comfort and decision execution ARRs of −188, −205, −15 and −184, respectively. These reward values are slightly lower than those of DulDQN + OP2. However, in the three-lane scenarios, the overall ARR of DulDQN + OP1 converges to −1450, which deviates significantly compared to −687 achieved by DulDQN + OP2. It indicates that the point-level reward function adopted in DulDQN + OP1 is insufficient to make an accurate instantaneous evaluation of driving decisions. Especially in the more complex three-lane scenarios, the features of the ending-time state point in the time horizon of the driving decision cannot fully reflect the effect of driving decision. Different decisions may correspond to the same state point features at high frequencies. Meanwhile, for the global state value evaluation, such inaccuracies would gradually accumulate, leading to the deteriorated learning performance of the state value function during training.

For both scenarios, the training curves of DulDQN + Poly show a convergence trend though with large fluctuations and undesirable learning performance. This is mainly caused by the inconsistency between the expected goals of its driving decision module and motion planning module. Polynomial path planning and IDM speed planning use mathematical models with fixed parameters, which cannot realize the expected state features reflected by the reward function in the driving decision model. In other words, the optimal motion planning results guided by each driving decision cannot be fed back to the driving decision module, thus impacting the learning performance of upper DRL driving decisions. Note that IDM in the DulDQN + Poly framework defines the expected headway guaranteeing

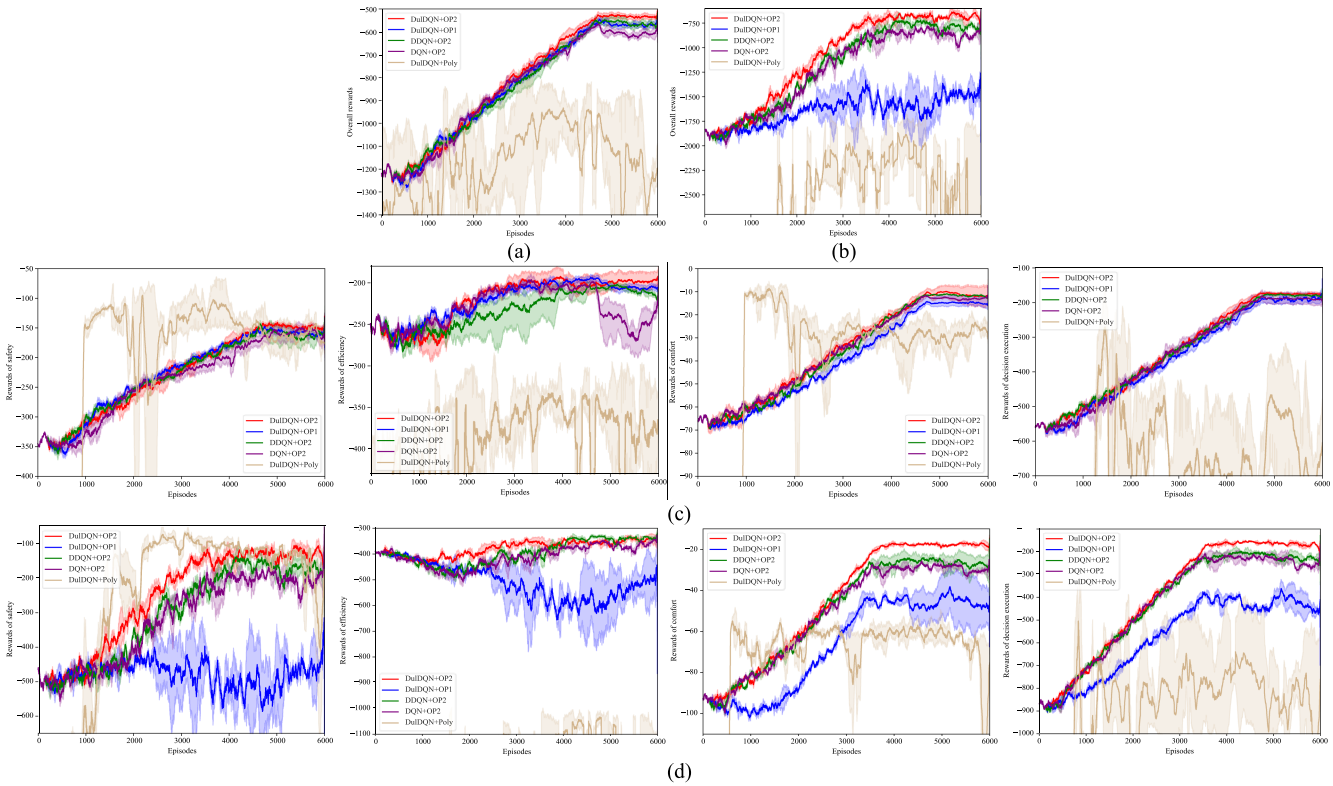

Fig. 5. Training results of all double-layer RL frameworks. (a) Overall reward in two-lane scenarios. (b) Overall reward in three-lane scenarios. (a) Overall reward in two-lane scenarios. (b) Overall reward in three-lane scenarios. (d) Rewards for safety, efficiency, comfort and decision execution in three-lane scenarios.

the safety of vehicle driving and guiding the framework to converge in terms of safety reward. However, for efficiency, comfort and decision execution rewards, the lower-level motion planning module failed to define specific objectives, leading to poor convergence.

Taken together, the above comparison results help validate the training effectiveness of the proposed DulDQN + OP2 framework.

C. Two Typical Lane-Changing Cases

Two typical lane-changing cases in two-lane and three-lane scenarios were scrutinized below to further examine the performance of the double-layer RL framework.

1) Lane-Changing Case in Two-Lane Scenario: In this case, according to the initial setting in Section VI-A, four surrounding vehicles were generated with different velocities to randomly stagger over the two lanes. The performance of integrated decision-making and motion planning was compared between DulDQN + OP2 and IDM + MOBIL. The time-dependent positions of the ego and surrounding vehicles are shown in Fig. 6(a) and (b). The ego trajectories guided by DulDQN + OP2 and IDM + MOBIL are shown in Fig. 6(c). All surrounding vehicles were guided by IDM + MOBIL.

At time 0 s, the ego guided by DulDQN + OP2 and IDM + MOBIL started with a speed of 20 m/s in the same state. At time 9.1 s, the ego guided by DulDQN + OP2 started to change lanes to avoid slowing down in the current lane. Then, from 9.1 s to 10.7 s, the ego safely changed to another lane under the blocking of veh1 and veh2, and continuously increased its velocity toward the desired velocity (34 m/s). Meanwhile, veh1 also implemented lane-changing under the interacting of the ego. By comparison, the ego guided by IDM + MOBIL remained in the current lane and was blocked by veh2 during this process. Then, it decelerated to maintain a safe distance from veh2. At time 10.7s, IDM + MOBIL reached the lane-changing condition and started to change to another lane. Then, from time 10.7s to time 13.1s, the ego implemented the lane change by decelerating first and then accelerating. In this process, DulDQN + OP2 accelerated to shorten the gap with the expected speed. From time 14.1s to 19.4s, DulDQN + OP2 made lane changes twice, overtaking veh2 and veh3, respectively, whereas IDM + MOBIL maintained driving in the current lane and did not overtake any vehicle. During the travel, except for a slight reduction in velocity during lane changes, the ego guided by DulDQN + OP2 maintained an overall increasing trend in velocity. At time 21.5 s, the ego overtook all other vehicles, and the case ended for DulDQN + OP2. By comparison, the case continued for IDM + MOBIL. At time 23.6 s, the ego started the second lane change and completed it at time 26.5 s. Then, the ego kept driving in the current lane and continued to reduce its velocity to maintain a safe distance from veh4 ahead. The case ended for IDM + MOBIL when the maximum timestep (time 50 s) was reached.

The trajectories of the ego guided by these two frameworks during the entire driving process are shown in Fig. 6(c). DulDQN + OP2 made three lane changes with the average velocity of

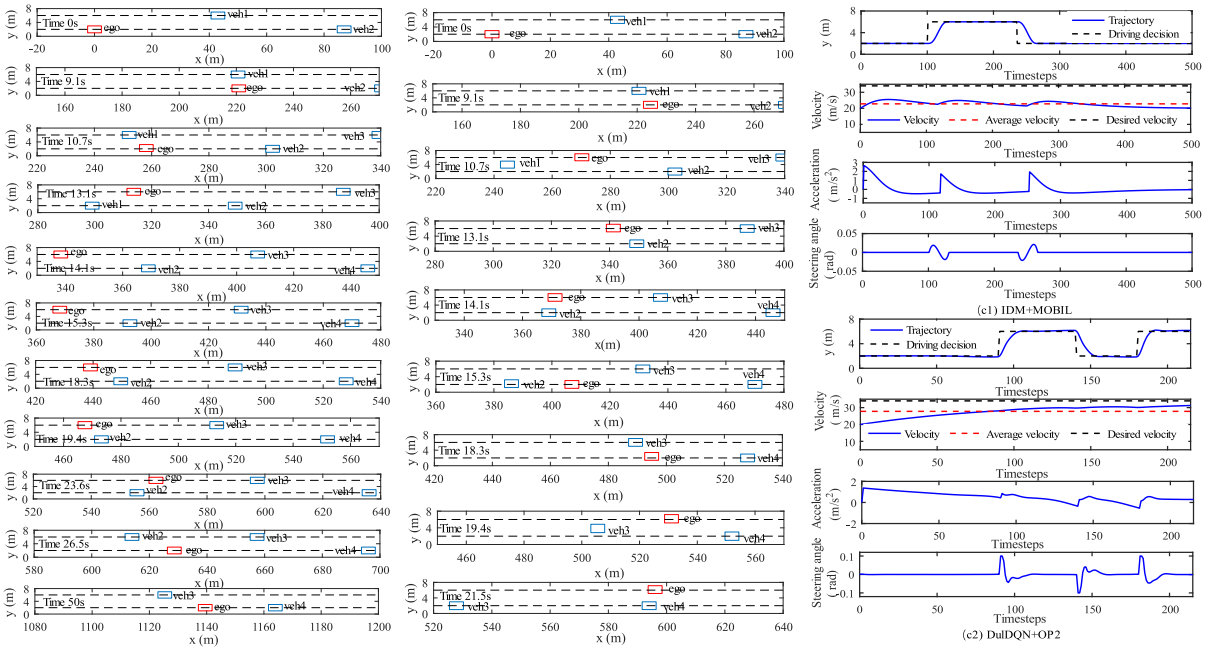

Fig. 6. Simulation results of the ego vehicle and surrounding vehicles in two-lane case. (a) Time-dependent positions by IDM+MOBIL. (b) Time-dependent positions by DulDQN+OP2. (c) Ego trajectories of these two frameworks.

TABLE III
PERFORMANCE EVALUATION RESULTS IN TWO-LANE CASE

Performance	IDM+MOBIL		DulDQN+OP2	
	Reward for one episode	Average reward for each timestep	Reward for one episode	Average reward for each timestep
Overall	-1904.49	-3.81	-520.66	-2.42
Safety	-206.25	-0.41	-142.74	-0.66
Efficiency	-1282.21	-2.56	-204.49	-0.95
Comfort	-20.33	-0.041	-10.72	-0.049
Decision execution	-395.71	-0.79	-162.40	-0.76

27.78 m/s, whereas IDM + MOBIL made two lane changes with an average velocity of 22.79 m/s. DulDQN + OP2 and IDM + MOBIL took 21.5 s (215 timesteps) and 50 s (500 timesteps), respectively, to complete the episode task. Their performance evaluation results are shown in Table III. It can be observed that IDM + MOBIL and DulDQN + OP2 achieved average safety rewards of -0.41 and -0.66 per timestep, respectively. The relatively lower speeds generated by IDM + MOBIL resulted in the ego suffering less driving risk. However, trajectory data analysis revealed that the minimum headway between the ego guided by DulDQN + OP2 and other surrounding vehicles was about 1.1s, which falls within the effective and safe headway range of 1s to 2s [33]. Furthermore, DulDQN + OP2 achieved an average efficiency reward of -0.95 per timestep, significantly higher than the -2.56 achieved by IDM + MOBIL. This indicates that compared to DulDQN + OP2, IDM + MOBIL performs relatively conservative in pursuing efficiency while maintaining safety. In the aspect of comfort, these two frameworks achieved similar average reward values per timestep, which were -0.041 for IDM + MOBIL and -0.049 for DulDQN + OP2. Besides, although DulDQN + OP2 implemented more times of lane changes than IDM + MOBIL, it obtained an average decision execution reward of -0.76 per timestep, which is close to -0.79 achieved by IDM + MOBIL. This is primarily due to the efficient execution of DulDQN + OP2, which helps shorten the lane-changing path and reduce the cost of each lane change.

Overall, DulDQN + OP2 attained the overall reward of -520.66, which is significantly higher than the -1904.49 achieved by IDM + MOBIL. In the aspects of safety, efficiency, comfort and decision execution, DulDQN + OP2 obtained rewards of -142.74, -204.49, -10.72 and -162.40, respectively, which are also higher than the rewards of -206.25, -1282.21, -20.33 and -395.71 achieved by IDM + MOBIL.

2) Lane-Changing Case in Three-Lane Scenario: Similarly, in the three-lane scenario the ego and 6 surrounding vehicles were randomly generated and distributed in three lanes. Fig. 7 shows the time-dependent positions and the ego trajectories by two frameworks.

The ego started with a speed of 15 m/s. During the overall driving process, the ego guided by DulDQN + OP2 implemented two lane changes at 0 s and 19.1 s, and overtook all surrounding vehicles at 26.5 s. By comparison, the ego guided by IDM + MOBIL implemented three lane changes at 2.1 s, 16.6 s and 27.1 s, respectively. Specifically, at time 0 s DulDQN + OP2 sensed the blocking of veh2 and the free space in the middle lane. Thus, the ego was guided to change to the middle lane immediately. At time 19.1s, although there exist veh4 in the left lane and a large driving space in the right lane, DulDQN + OP2 guided the ego to change to the left lane considering the potential impact of

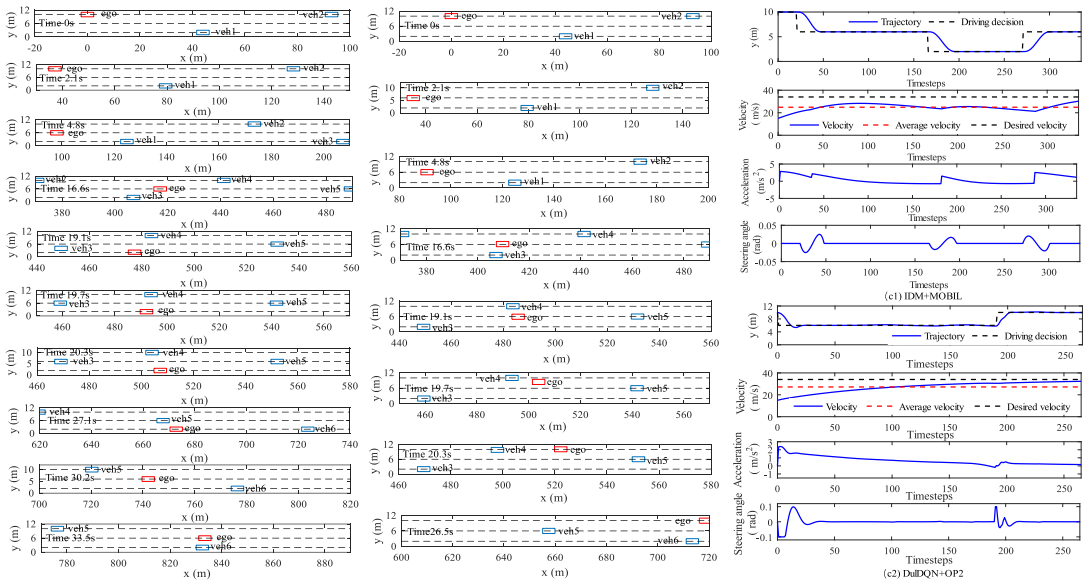

Fig. 7. Simulation results of the ego vehicle and surrounding vehicles in three-lane case. (a) Time-dependent positions by IDM+MOBIL. (b) Time-dependent positions by DulDQN+OP2. (c) Ego trajectories by these two frameworks.

the slow-moving veh6 ahead in the right lane. Thus, it enabled the ego to overtake all surrounding vehicles with fewer lane changes. In contrast, IDM + MOBIL only focused on the current positions of the ego and surrounding vehicles when making driving decisions. At time 16.6s, since the vehicle states in the right lane satisfied the lane-changing conditions of MOBIL, the ego immediately changed to the right lane without expecting the negative impact of slow-moving veh6 ahead. Then, when the ego arrived at the right lane, the acceleration maneuver just taken gradually turned to deceleration until the ego finally returned to the middle lane to avoid veh6. These observations imply that IDM + MOBIL tends to focus on the local optimization of the current state and fails to account for vehicle dynamics in the future. In contrast, DulDQN + OP2 was trained based on the evaluation of the global states. Thus, the driving decisions made represent the global optimal results, taking into account their impact on both the current state and the future states.

DulDQN + OP2 took 26.5 s (265 timesteps) to complete the episode task with an average speed of 27.17 m/s, which is more efficient than the 33.5 s (335 timesteps) and 24.90 m/s achieved by IDM + MOBIL. The detailed performance evaluation results are shown in Table IV. The average overall reward per timestep for DulDQN + OP2 is −2.15, which is higher than −4.02 achieved by IDM + MOBIL. In addition, the rewards achieved by DulDQN + OP2 in the aspects of safety, efficiency, comfort and decision execution per timestep are −0.23, −1.34, −0.04 and −0.54, respectively, which are also higher than −0.32, −1.80, −0.15 and −1.75 obtained by IDM + MOBIL. The global optimization ability of DulDQN + OP2 allows it to avoid generating redundant decisions and effectively utilize the limited time and road resources.

The average solution time of DRL driving decision and MPC motion planning is 0.42 ms and 24.24 ms per time step, respectively. According to Claussmann et al. [34], the proposed framework can meet the real-time application requirements of AD system.

TABLE IV
PERFORMANCE EVALUATION RESULTS IN THREE-LANE CASE

Performance	IDM+MOBIL		DulDQN+OP2	
	Reward for one episode	Average reward for each timestep	Reward for one episode	Average reward for each timestep
Overall	-1346.94	-4.02	-569.70	-2.15
Safety	-105.87	-0.32	-61.23	-0.23
Efficiency	-604.34	-1.80	-355.27	-1.34
Comfort	-51.19	-0.15	-11.42	-0.04
Decision execution	-585.54	-1.75	-141.80	-0.54

D. Comparison With Empirical Driving and Other Frameworks

The experiments were further conducted based on the NGSIM (Next Generation Simulation) naturalistic human driving datasets. Based on the NGSIM US101 database, vehicle trajectories were first processed by noise filtering, extraction, and spatiotemporal alignment, and then driving environment was created including both road and environmental vehicles. In the driving environment, we arbitrarily selected one vehicle as the ego, and other vehicles within a certain range (±100m) from the ego during the entire driving period as obstacles. The proposed framework aims to control the interaction between the ego and other human-driven vehicles to achieve safe and efficient driving. A comprehensive evaluation of the proposed framework was conducted via 100 driving experiments involving the ego with different Vehicle_IDs. Furthermore, the scenario involving

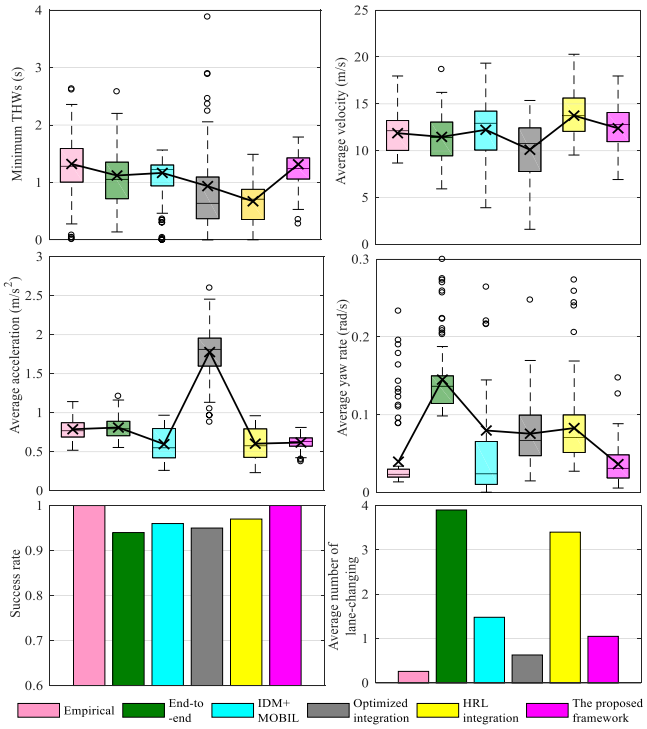

Fig. 8. Comprehensive evaluation results of vehicle trajectories in 100 driving experiments.

Vehicle_ID-1740 was chosen as a representative case to compare the generated vehicle trajectory by the framework with the trajectory of human driving.

The proposed DulDQN + OP2 framework was compared with four benchmark frameworks, including end-to-end scheme [18], IDM + MOBIL decomposition scheme [35], the optimization-based integration scheme [26] and the Hierarchical Reinforcement Learning (HRL) based integration scheme [27]. we conducted comparative analyses using commonly used vehicle trajectory features, including average speed, minimum time headway (THW), average acceleration, average yaw rate and average lane-changing frequency.

Fig. 8 shows the comprehensive evaluation results of vehicle trajectories in 100 driving experiments. It can be observed that the proposed framework achieves an average speed of 12.39 m/s for each trajectory, which is higher than 11.86 m/s achieved by empirical driving, 11.6 m/s by the end-to-end scheme, 12.20 m/s by IDM + MOBIL, and 10.09 m/s by the optimized integration framework. Although the average speed by the proposed framework is lower than 13.74 m/s achieved by the HRL integration framework, the safety performance by the HRL integration framework is the worst, with an average minimum THW of 0.67 s. This can be attributed to two reasons: 1) the sparse point-level reward function in the upper-level driving decision model of the HRL integration framework is set more aggressively; 2) its longitudinal acceleration action space is discrete with poor flexibility of speed adjustment, making it easy to drive excessively aggressively or conservatively. The average minimum THW achieved by the proposed framework is 1.317 s, which is close to 1.322 s of the empirical trajectories, and higher than 1.112 s achieved by the end-to-end scheme, 1.167 s by IDM + MOBIL, and 0.937 s by the optimized integration framework. These results indicate that compared to the empirical trajectories and the benchmarks the proposed framework enables to achieve the most desirable safety performance while maintaining driving efficiency.

TABLE V
STATISTICAL RESULTS OF VEHICLE TRAJECTORY IN THE SCENARIO OF VEHICLE-ID 1740

Method	Average velocity (m/s)	Minimum THWs (s)	Average acceleration (m/s^2)	Average yaw rate (rad/s)	Number of lane changes
Empirical	12.704	0.799	0.634	0.022	0
End-to-end	12.844	0.520	0.682	0.132	4
IDM+ MOBIL	12.636	0.798	0.370	0.053	2
Optimized integration	12.983	0.388	2.154	0.145	0
HRL integration	15.440	0.280	0.684	0.073	4
The proposed framework	**14.059**	**1.033**	**0.414**	**0.020**	**1**

Additionally, the average acceleration by the proposed framework is 0.62m/s^2, which is slightly higher than 0.596 m/s^2 achieved by the IDM + MOBIL framework and 0.60 m/s^2 by the HRL integration framework, but lower than 0.79 m/s^2 achieved by the empirical driving, 0.81 m/s^2 by the end-to-end solution, and 1.78 m/s^2 by the optimized integration framework. It is worth noting that the optimized integration framework fails to achieve desirable results in terms of safety, efficiency and longitudinal driving comfort. This may be due to the fact that its upper-level driving decision adopts a predefined A$*$ global static path, which does not change with the dynamic environment. In practice, when a vehicle needs to change lanes to pursue higher safety or efficiency in the dynamic environment, this static driving decision may form unreasonable constraints on vehicle motion control, thereby affecting the safety and efficiency of the vehicle. This, in turn, leads to frequent vehicle deceleration or acceleration in order to safely avoid obstacles or pursue higher speeds. For the yaw rate, it can be found that the average yaw rate by the proposed framework is close to the empirical driving trajectory and superior to the other four frameworks. The average yaw rate by the end-to-end scheme is the lowest, which may be due to the inability of the end-to-end scheme to perform comfort constraints or its failure to learn the desirable effect.

The results of success rate further demonstrate the safety performance by the proposed framework. According to the statistics of lane-changing frequency, the average number of lane changes per trajectory by the proposed framework is 1.05, which is lower than those by the IDM + MOBIL framework, the end-to-end framework, and the HRL integration framework. It indicates that the DRL driving decision model trained by the proposed framework does not perform lane-changing frequently.

Fig. 9 and Table V show the vehicle trajectory results in the scenario of Vehicle-ID 1740. The empirical driving and the optimized integration framework exhibit relatively conservative driving behavior by maintaining the lane throughout the entire driving process. The HRL integration framework performed lane

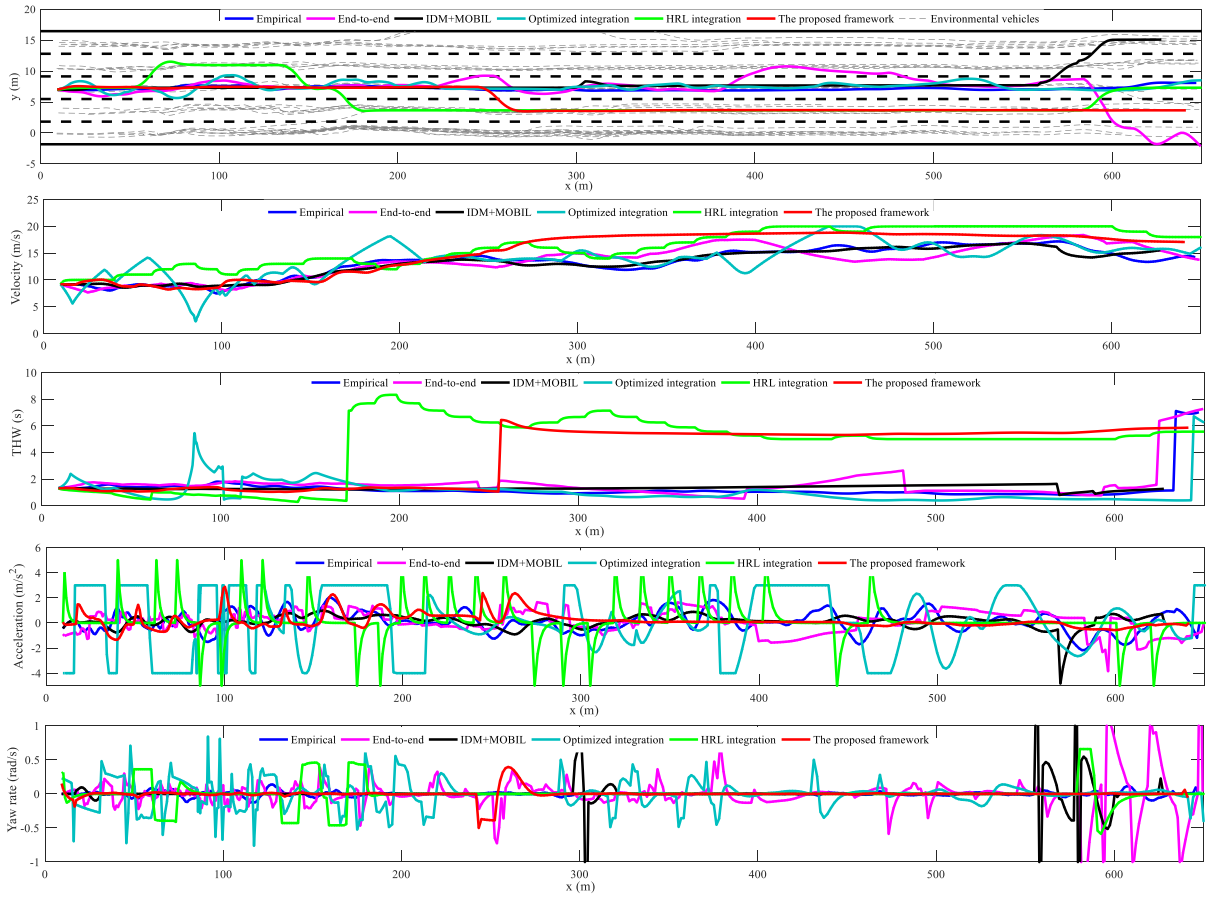

Fig. 9. Vehicle trajectories in the scenario of vehicle-ID 1740.

changes 4 times with the highest speed and worst safety performance in an aggressive manner. By comparison, the proposed framework identified a lane-changing opportunity in the right lane near x = 250 m and executed an acceleration maneuver, thereby enhancing the overall driving efficiency. The trajectory features curve in Fig. 9 and the statistical results in Table V reveal that the average velocity by the proposed framework in this scenario is 14.059 m/s, which is higher than those by the empirical trajectory, the end-to-end scheme, the IDM + MOBIL framework, and the optimized integration framework. Although the HRL integration framework achieves higher trajectory efficiency than the proposed framework, its safety performance is the worst, with a minimum THW value of 0.280 s. By comparison, the proposed framework achieves the most desirable safety performance, with a minimum THW value of 1.033 s. The average acceleration, and yaw rate results also indicate that the proposed framework achieves the desirable safety performance and comfort.

In summary, the proposed framework enables the desirable comprehensive performance compared to empirical driving and the other four frameworks in terms of safety, efficiency, and comfort. It also helps demonstrate the effectiveness of closed-loop optimization mechanism formed by the upper-level DRL global driving decision guidance and the lower-level MPC local motion control feedback.

VII. Conclusion

In this study, a double-layer RL framework was proposed for autonomous vehicle to integrate driving decision-making and motion planning modules. In the upper layer, a trajectory-level reward function in regard to safety, efficiency, comfort and decision execution was established. Furthermore, through the design of state space, action space and network structure, the driving decision models were developed using three DQN series value-based DRL algorithms (DQN, DDQN and Dueling DQN). In the lower layer, a constrained motion planning model based on MPC was established, with the trajectory-level reward function as the objective function to ensure the integration compatibility and goal consistency with the upper-level driving decision-making model. Using this framework, the DRL decision-making model outputs driving decisions to guide the optimal trajectory planning of the lower level. Meanwhile, the guided vehicle travels along the trajectory to update the states and feeds the trajectory evaluation results back to the upper decision-making layer for parameter optimization. Then, the proposed framework was trained and tested in two-lane and three-lane scenarios. The results demonstrated that the proposed double-layer RL frameworks outperformed DulDQN + Poly and IDM + MOBIL in terms of overall, safety, efficiency, comfort and decision execution rewards. Furthermore, among the three RL frameworks DulDQN + OP2 outperformed the

counterparts in overall reward and stability. The effectiveness of the trajectory-level reward function was also verified in contrast to the state point-level reward function. Finally, the NGSIM driving environment was established to further validate the effectiveness and practicality of the proposed framework. The results indicated that the proposed framework could adapt to human driving environments and outperform empirical driving trajectories in terms of safety, efficiency and comfort. Comparative experiments with four benchmark frameworks also demonstrated the superiority of our proposed framework in comprehensive performance. All these analyses help verify that the proposed framework has the potential to be applied in the AD system.

In the future, the framework can be further explored in the following aspects. First, the extension of this framework under multiple complex scenarios will be investigated. Next, we will calibrate the weight coefficients of the reward function based on real natural driving data, and explore the driving performance corresponding to different driving styles. Furthermore, the trajectory prediction of surrounding vehicles will assist in promoting the safety performance of the framework.

REFERENCES

[1] C. Zhao, L. Li, X. Pei, Z. Li, F.-Y. Wang, and X. Wu, "A comparative study of state-of-the-art driving strategies for autonomous vehicles," *Accident Anal. Prevention*, vol. 150, Dec. 2021, Art. no. 105937.

[2] G. Singh et al., "Road: The road event awareness dataset for autonomous driving," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 1, pp. 1036–1054, Jan. 2023.

[3] W. Wang et al., "Decision-making in driver-automation shared control: A review and perspectives," *IEEE/CAA J. Automatica Sinica*, vol. 7, no. 5, pp. 1289–1307, Sep. 2020.

[4] C. Lei, "Deep reinforcement learning," in *Deep Learning and Practice With Mind Spore*. Berlin, Germany: Springer, 2021, pp. 217–243.

[5] Y. Liang, Y. Li, A. Khajepour, Y. Huang, Y. Qin, and L. Zheng, "A novel combined decision and control scheme for autonomous vehicle in structured road based on adaptive model predictive control," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 9, pp. 16083–16097, Sep. 2022.

[6] W. Wang, T. Qie, C. Yang, W. Liu, C. Xiang, and K. Huang, "An intelligent lane-changing behavior prediction and decision-making strategy for an autonomous vehicle," *IEEE Trans. Ind. Electron.*, vol. 69, no. 3, pp. 2927–2937, Mar. 2022.

[7] A. Kesting, M. Treiber, and D. Helbing, "General lane-changing model MOBIL for car-following models," *Transp. Res. Rec.*, vol. 1999, no. 1, pp. 86–94, Jan. 2007.

[8] Y. Ali, Z. Zheng, M. M. Haque, and M. Wang, "A game theory-based approach for modelling mandatory lane-changing behaviour in a connected environment," *Transp. Res. Part C: Emerg. Technol.*, vol. 106, pp. 220–242, Sep. 2019.

[9] M. Kamrani, A. R. Srinivasan, S. Chakraborty, and A. J. Khattak, "Applying Markov decision process to understand driving decisions using basic safety messages data," *Transp. Res. Part C: Emerg. Technol.*, vol. 115, Jun. 2020, Art. no. 102642.

[10] C. Vallon, Z. Ercan, A. Carvalho, and F. Borrelli, "A machine learning approach for personalized autonomous lane change initiation and control," in *Proc. IEEE Intell. Veh. Symp. 4th*, 2017, pp. 1590–1595.

[11] J. Morton, T. A. Wheeler, and M. J. Kochenderfer, "Analysis of recurrent neural networks for probabilistic modeling of driver behavior," *IEEE Trans. Intell. Transp. Syst.*, vol. 18, no. 5, pp. 1289–1298, May 2017.

[12] S. J. Anderson, S. B. Karumanchi, and K. Iagnemma, "Constraint-based planning and control for safe, semi-autonomous operation of vehicles," in *Proc. IEEE Intell. Veh. Symp. 4th*, 2012, pp. 383–388.

[13] O. Pink, C. Frese, and C. Stiller, "Team AnnieWAY's autonomous system for the 2007 DARPA urban challenge," *J. Field Robot.*, vol. 25, no. 9, pp. 615–639, Aug. 2010.

[14] C. Katrakazas, M. Quddus, W. H. Chen, and L. Deka, "Real-time motion planning methods for autonomous on-road driving: State-of-the-art and future research directions," *Transp. Res. Part C: Emerg. Technol.*, vol. 60, pp. 416–442, Nov. 2015.

[15] J. Chen, P. Zhao, T. Mei, and H. Liang, "Lane change path planning based on piecewise Bezier curve for autonomous vehicle," in *Proc. IEEE Int. Conf. Veh. Electron. Saf.*, 2013, pp. 17–22.

[16] S. Glaser, B. Vanholme, S. Mammar, D. Gruyer, and L. Nouveliere, "Maneuver-based trajectory planning for highly autonomous vehicles on real road with traffic and driver interaction," *IEEE Trans. Intell. Transp. Syst.*, vol. 11, no. 3, pp. 589–606, Sep. 2010.

[17] B. Wang et al., "Interpretable motion planner for urban driving via hierarchical imitation learning," 2023, *arXiv:2303.13986*.

[18] A. Kuefler, J. Morton, T. Wheeler, and M. Kochenderfer, "Imitating driver behavior with generative adversarial networks," in *Proc. IEEE Intell. Veh. Symp. 4th*, 2017, pp. 204–211.

[19] C. Liu, S. Lee, S. Varnhagen, and H. E. Tseng, "Path planning for autonomous vehicles using model predictive control," in *Proc. IEEE Intell. Veh. Symp. 4th*, 2017, pp. 174–179.

[20] S. Dixit et al., "Trajectory planning for autonomous high-speed overtaking in structured environments using robust MPC," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 6, pp. 2310–2323, Jun. 2020.

[21] M. Bojarski et al., "End to end learning for self-driving cars," 2016, *arXiv: 1604.07316v1*.

[22] F. Codevilla, M. Müller, A. López, V. Koltun, and A. Dosovitskiy, "End-to-end driving via conditional imitation learning," in *Proc. IEEE Int. Conf. Robot. Automat.,* 2018, pp. 4693–4700.

[23] X. Tang, B. Huang, T. Liu, and X. Lin, "Highway decision-making and motion planning for autonomous driving via soft actor-critic," *IEEE Trans. Veh. Technol.*, vol. 71, no. 5, pp. 4706–4717, May 2022.

[24] G. Li, Y. Yang, S. Li, X. Qu, N. Lyu, and S. E. Li, "Decision making of autonomous vehicles in lane change scenarios: Deep reinforcement learning approaches with risk awareness," *Transp. Res. Part C: Emerg. Technol.*, vol. 134, Jan. 2022, Art. no. 103452.

[25] P. Hang, C. Lv, C. Huang, J. Cai, Z. Hu, and Y. Xing, "An integrated framework of decision making and motion planning for autonomous vehicles considering social behaviors," *IEEE Trans. Veh. Technol.*, vol. 69, no. 12, pp. 14458–14469, Dec. 2020.

[26] H. Liu, K. Chen, Y. Li, Z. Huang, J. Duan, and J. Ma, "Integrated decision-making and control for urban autonomous driving with traffic rules compliance," 2023, *arXiv:2304.01041*.

[27] Y. Lu, X. Xu, X. Zhang, L. Qian, and X. Zhou, "Hierarchical reinforcement learning for autonomous decision making and motion planning of intelligent vehicles," *IEEE Access*, vol. 8, pp. 209776–209789, 2020.

[28] C. J. Hoel, K. Wolff, and L. Laine, "Automated speed and lane change decision making using deep reinforcement learning," in *Proc. 21st Int. Conf. Intell. Transp. Syst.*, 2018, pp. 2148–2155.

[29] S. Li, C. Wei, and Y. Wang, "Combining decision making and trajectory planning for lane changing using deep reinforcement learning," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 9, pp. 16110–16136, Sep. 2022.

[30] M. Ammour, R. Orjuela, and M. Basset, "A MPC combined decision making and trajectory planning for autonomous vehicle collision avoidance," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 12, pp. 24805–24817, Dec. 2022.

[31] Y. Rasekhipour, A. Khajepour, S.-K. Chen, and B. Litkouhi, "A potential field-based model predictive path-planning controller for autonomous road vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 18, no. 5, pp. 1255–1267, May 2017.

[32] W. Wang, W. Wang, J. Wan, D. Chu, Y. Xu, and L. Lu, "Potential field based path planning with predictive tracking control for autonomous vehicles," in *Proc. 5th Int. Conf. Transp. Inf. Saf.*, 2019, pp. 746–751.

[33] M. Zhu, Y. Wang, Z. Pu, J. Hu, X. Wang, and R. Ke, "Safe, efficient, and comfortable velocity control based on reinforcement learning for autonomous driving," *Transp. Res. Part C: Emerg. Technol.*, vol. 117, Aug. 2020, Art. no. 102662.

[34] L. Claussmann, M. Revilloud, D. Gruyer, and S. Glaser, "A review of motion planning for highway autonomous driving," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 5, pp. 1826–1848, May 2020.

[35] F. A. Mullakkal-Babu, M. Wang, B. van Arem, B. Shyrokau, and R. Happee, "A hybrid submicroscopic-microscopic traffic flow simulation framework," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 6, pp. 3430–3443, Jun. 2021.

Yaping Liao received the M.S. degree from the Shandong University of Science and Technology, Qingdao, China, in 2019. She is currently working toward the Ph.D. degree with the School of Transportation Science and Engineering, Beihang University, Beijing, China. Her research interests include intelligent vehicle, decision making, and deep reinforcement learning.

Guizhen Yu received the Ph.D. degree from Jilin University, Changchun, China, in 2003. He is currently a Full Professor with the School of Transportation Science and Engineering, Beihang University, Beijing, China. His research interests include intelligent vehicle, urban traffic operation, and intelligent transportation systems.

Peng Chen (Member, IEEE) received the Ph.D. degree from Nagoya University, Nagoya, Japan, in 2012. He is currently an Associate Professor with the School of Transportation Science and Engineering, Beihang University, Beijing, China. His research interests include intelligent transportation systems, urban traffic operation and control, traffic flow modeling and simulation. He is the Member of IEEE ITS Society.

Bin Zhou received the Ph.D. degree from Beihang University, Beijing, China, in 2018. He is currently an Assistant Professor with the School of Transportation Science and Engineering, Beihang University, Beijing, China. His research interests include deep-learning algorithms and intelligent connected vehicles.

Han Li received the B.S. degree from Northeast Forestry University, Harbin, China, in 2018. He is currently working toward the Ph.D. degree with the School of Transportation Science and Engineering, Beihang University, Beijing, China. His research interests include decision making, trajectory planning, and trajectory tracking control of intelligent vehicles.